



■ CS177 Bioinformatics: Software Development and Applications

Lecture 8: Introduction to Deep Learning

Part 3. Graph Neural Network (GNN)

Jie Zheng (郑杰)

PhD, Associate Professor

School of Information Science and Technology (SIST), ShanghaiTech University

Spring, 2024

Slides mainly from: Thomas Kipf (University of Amsterdam), Bryan Perozzi (Google) KDD 2018 Tutorial



Learning objectives



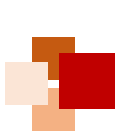
- After taking this lecture, you should be able to:
 - Point out the motivation for graph neural networks (GNNs)
 - Explain the main idea of graph convolutional networks (GCNs)
 - In regard to limitations of GCNs, describe a few extensions of GCNs (e.g. GNN with attention)
 - List and explain three main categories of applications of GNNs (node classification, graph classification, link prediction)
 - Given a specific application problem (e.g. in drug discovery), pick and try suitable GNN models





Graph-Structured Data





The success story of deep learning

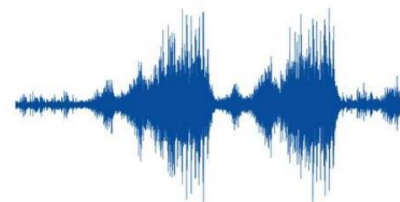


上海科技大学
ShanghaiTech University

IMAGENET

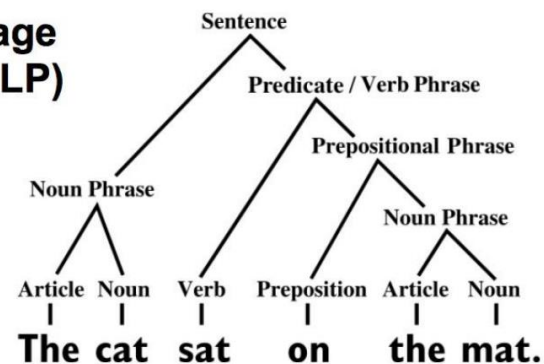


Speech data

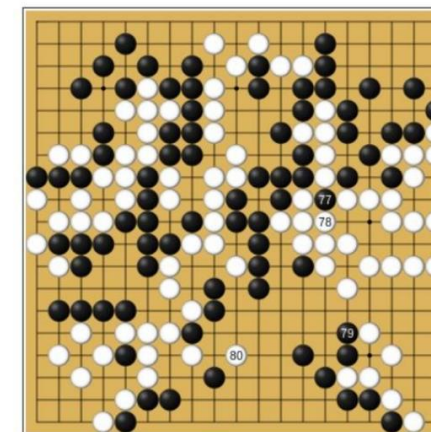


Natural language processing (NLP)

...

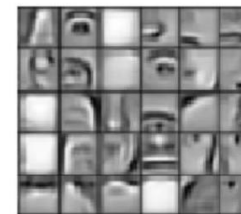
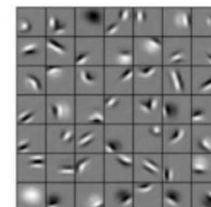


Grid games

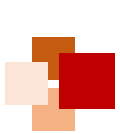


Deep neural nets that exploit:

- translation equivariance (weight sharing)
- hierarchical compositionality



立志成才 报國裕民

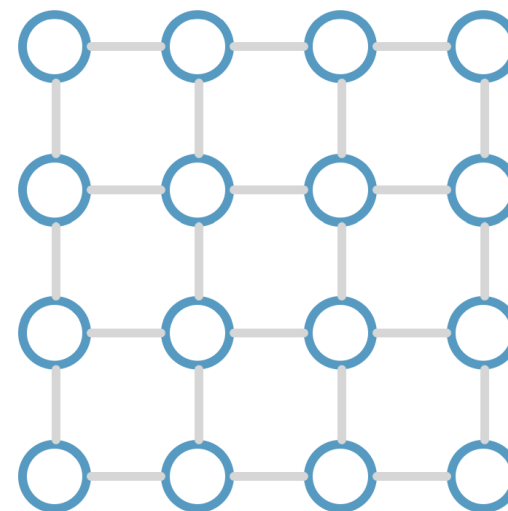
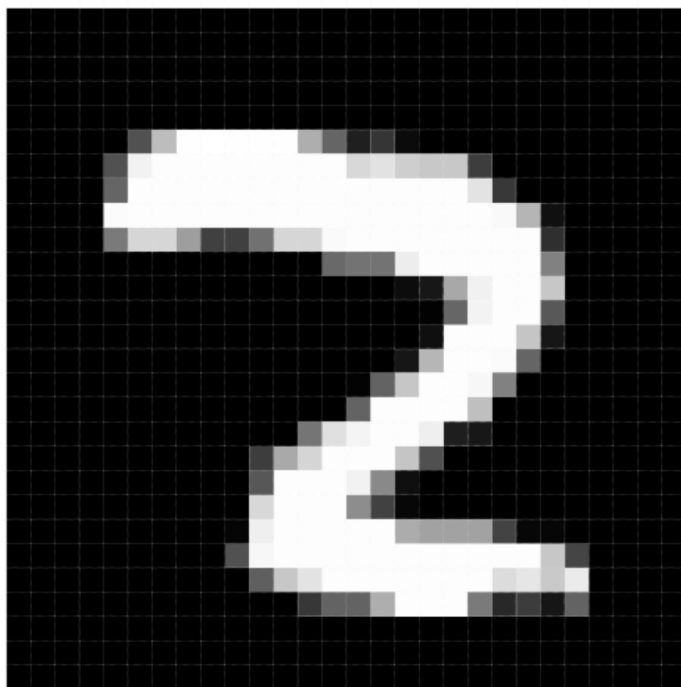


Recap: Deep learning on Euclidean data

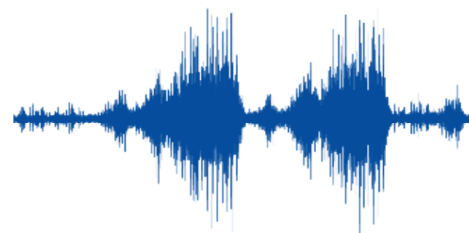


上海科技大学
ShanghaiTech University

Euclidean data: grids, sequences...



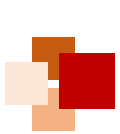
2D grid



1D grid



立志成才 报國裕民

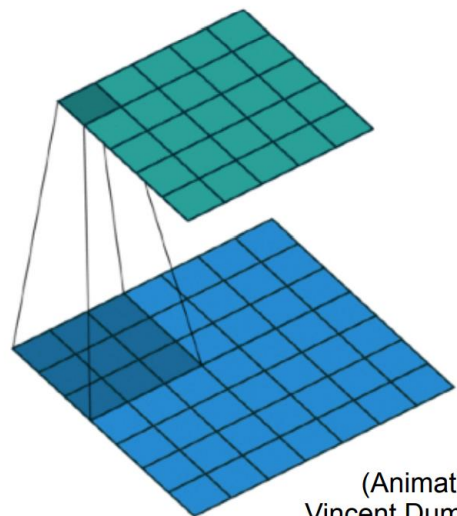


Recap: Deep learning on Euclidean data

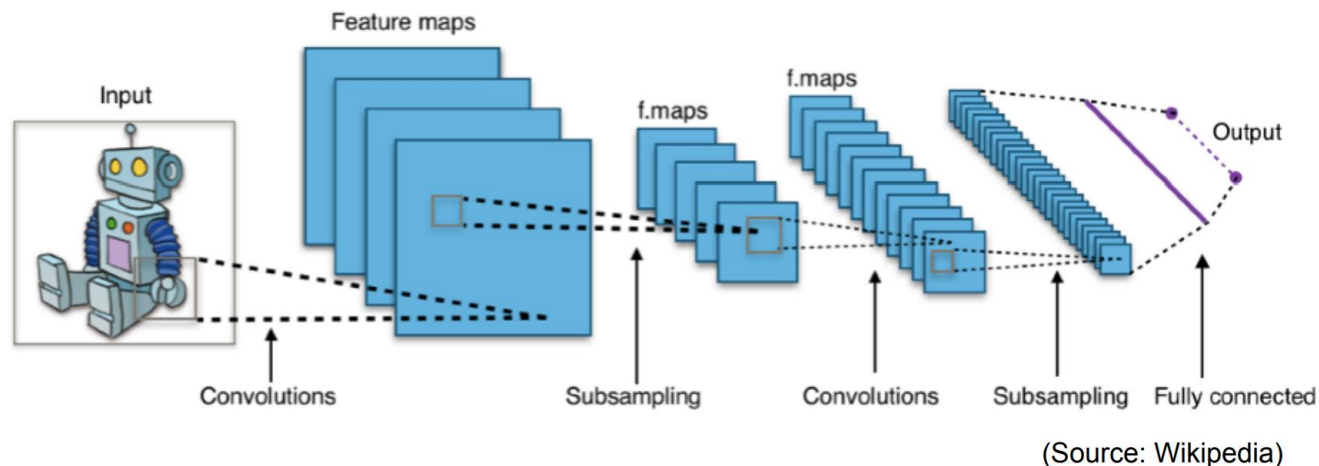


上海科技大学
ShanghaiTech University

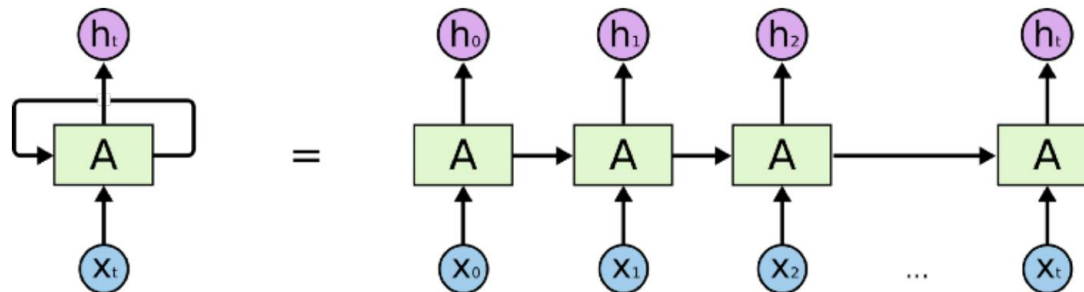
Convolutional neural networks (CNNs)



(Animation by
Vincent Dumoulin)



Recurrent neural networks (RNNs)



(Source: Christopher Olah's blog)

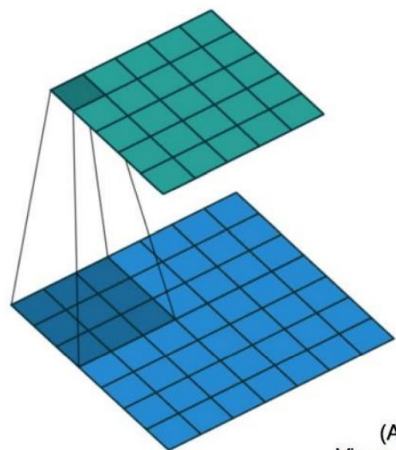


立志成才 报國裕民

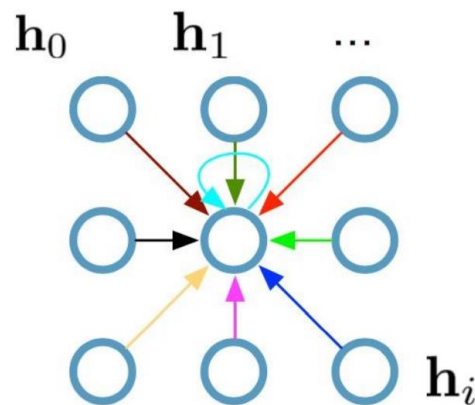
Recap: CNNs on Grids



Single CNN layer with 3x3 filter:



(Animation by
Vincent Dumoulin)



Update for a single pixel:

- Transform messages individually $\mathbf{W}_i \mathbf{h}_i$
- Add everything up $\sum_i \mathbf{W}_i \mathbf{h}_i$

$\mathbf{h}_i \in \mathbb{R}^F$ are (hidden layer) activations of a pixel/node

Full update:

$$\mathbf{h}_4^{(l+1)} = \sigma \left(\mathbf{W}_0^{(l)} \mathbf{h}_0^{(l)} + \mathbf{W}_1^{(l)} \mathbf{h}_1^{(l)} + \cdots + \mathbf{W}_8^{(l)} \mathbf{h}_8^{(l)} \right)$$

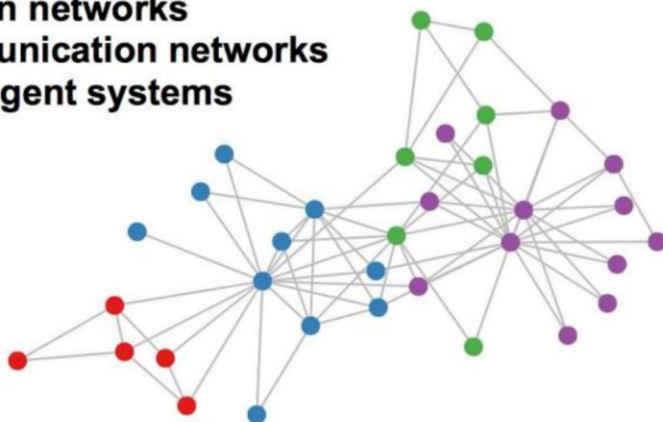


Graph-structured data

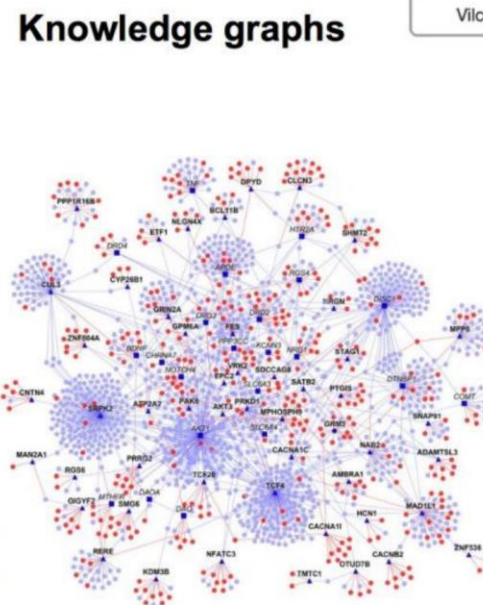


A lot of real-world data does not “live” on grids

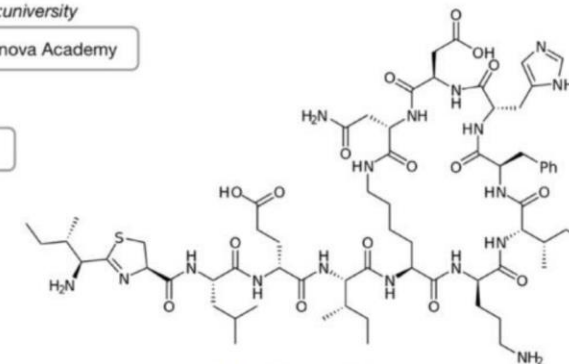
Social networks
Citation networks
Communication networks
Multi-agent systems



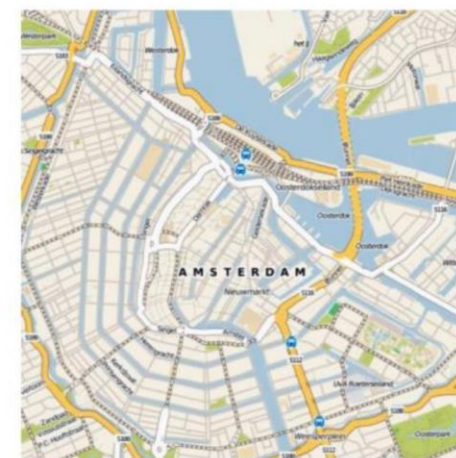
Protein interaction
networks



Knowledge graphs



Molecules



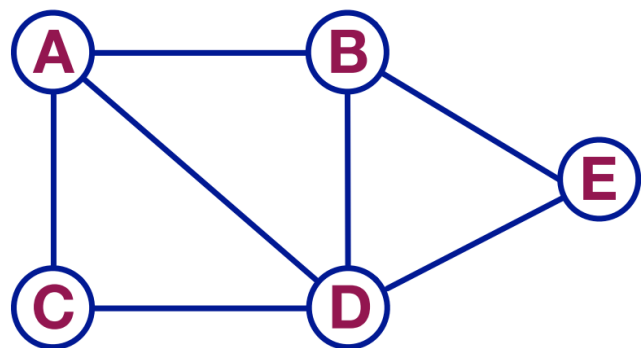
Road maps

Standard **CNN** and **RNN** architectures don't work on this data

Graph-structured data



Graph: $G = (\mathcal{V}, \mathcal{E})$



Adjacency matrix: A

	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	1
C	1	0	0	1	0
D	1	1	1	0	1
E	0	1	0	1	0

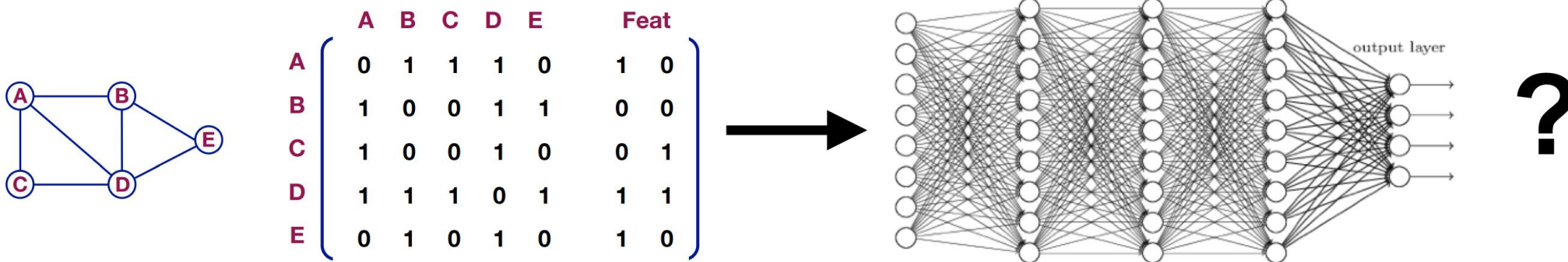
Model wish list:

- Trainable in $\mathcal{O}(|\mathcal{E}|)$ time
- Applicable even if the input graph changes



A naïve approach

- Take adjacency matrix $\mathbf{A}$ and feature matrix $\mathbf{X}$
- Concatenate them $\mathbf{X}_{\text{in}} = [\mathbf{A}, \mathbf{X}]$
- Feed them into deep (fully connected) neural net
- Done?



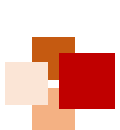
Problems:

- Huge number of parameters $\mathcal{O}(N)$
- Re-train if graph changes

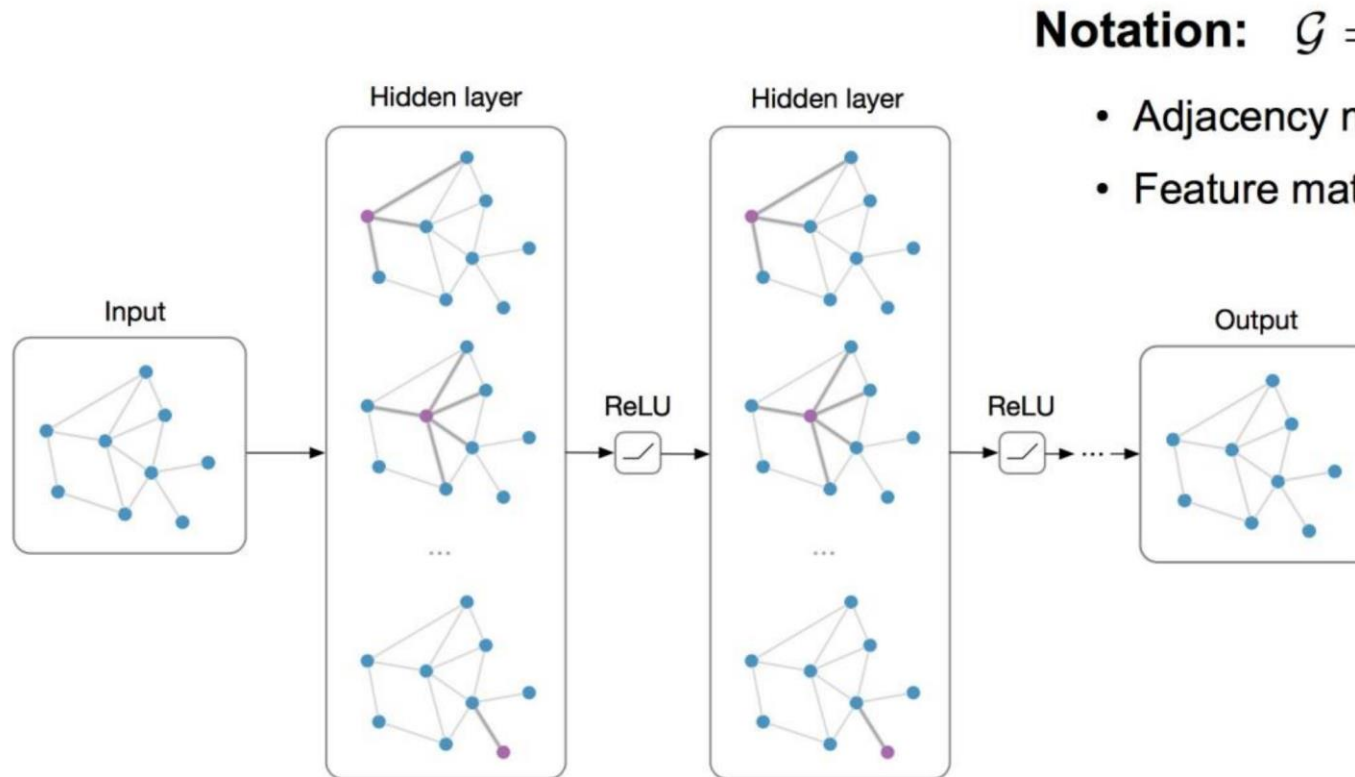


Graph Neural Networks (GNNs)





Graph neural networks (GNNs)

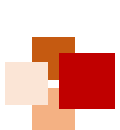


Notation: $\mathcal{G} = (\mathbf{A}, \mathbf{X})$

- Adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$
- Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times F}$

- **Main idea:** Pass messages between pairs of nodes and agglomerate
- **Alternative Interpretation:** Pass messages between nodes to refine the representation of nodes and edges

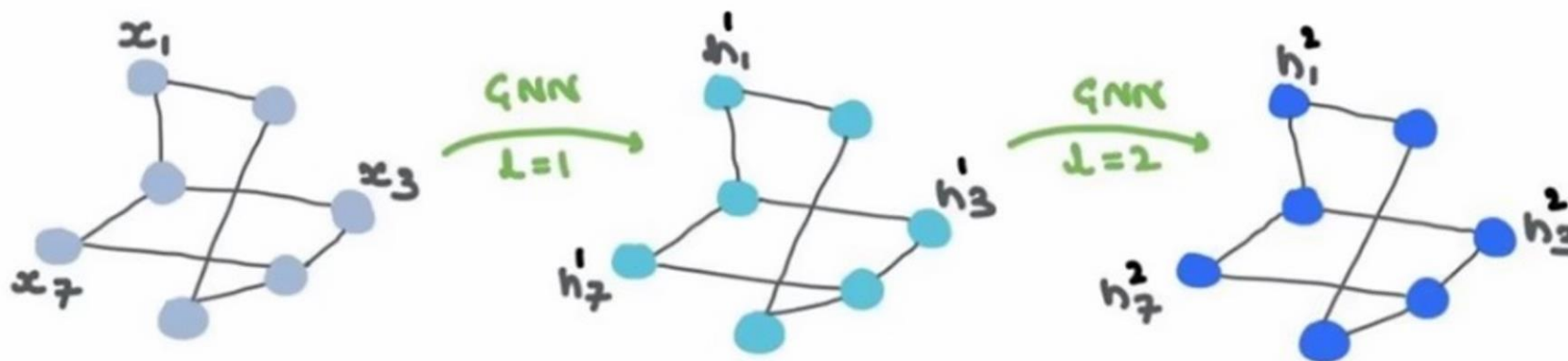


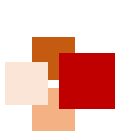


Graph neural networks (GNNs)



- GNNs can also have multiple layers
 - The essence of GNN is to update various feature components
 - Both the input and the output are features, and the adjacency matrix remains unchanged
- What can we do with the output features?
 - Can use the combined features of various points for graph classification
 - Can classify individual nodes or edges
 - In reality, it is just about extracting features from the graph structure; what we ultimately do with them depends on our own objectives





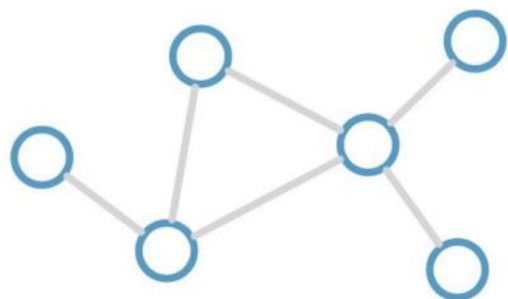
Graph Convolutional Networks (GCNs)



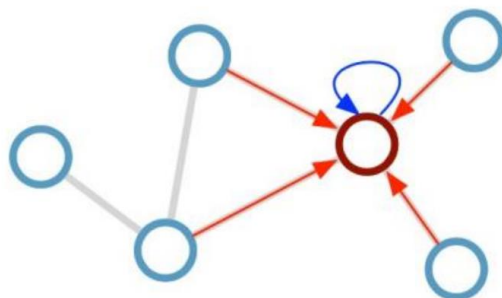
上海科技大学
ShanghaiTech University

Kipf & Welling (ICLR 2017), related previous works by Duvenaud et al. (NIPS 2015) and Li et al. (ICLR 2016)

Consider this undirected graph:



Calculate update for node in red:



Update rule:

$$\mathbf{h}_i^{(l+1)} = \sigma \left(\mathbf{h}_i^{(l)} \mathbf{W}_0^{(l)} + \sum_{j \in \mathcal{N}_i} \frac{1}{c_{ij}} \mathbf{h}_j^{(l)} \mathbf{W}_1^{(l)} \right)$$

Scalability: subsample messages [Hamilton et al., NIPS 2017]

Desirable properties:

- Weight sharing over all locations
- Invariance to permutations
- Linear complexity $O(E)$
- Applicable both in transductive and inductive settings

GCNs with first-order message passing

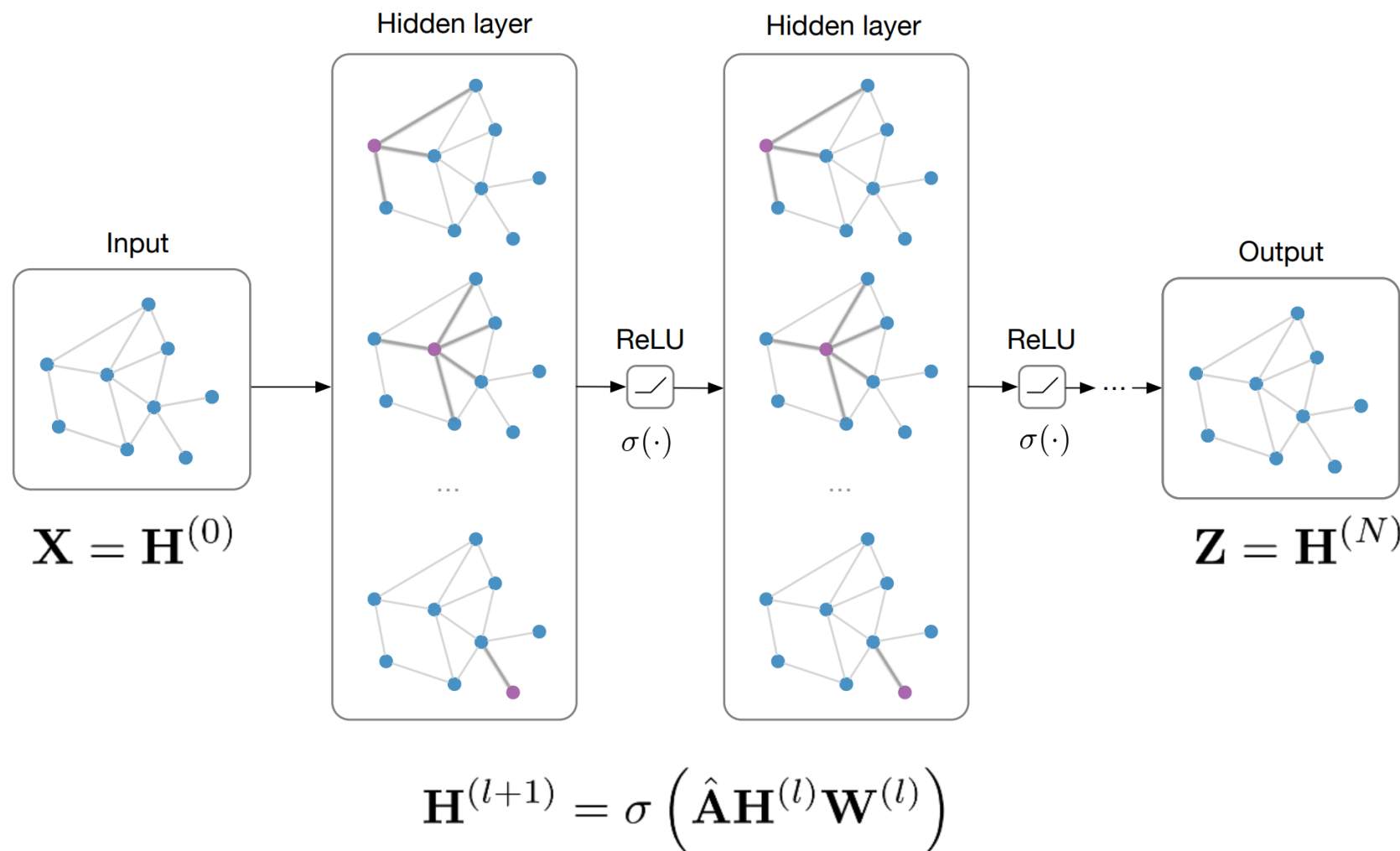
$\mathcal{N}_i$: neighbor indices c_{ij} : norm. constant (fixed/trainable)



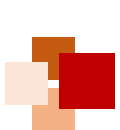
GCN model architecture



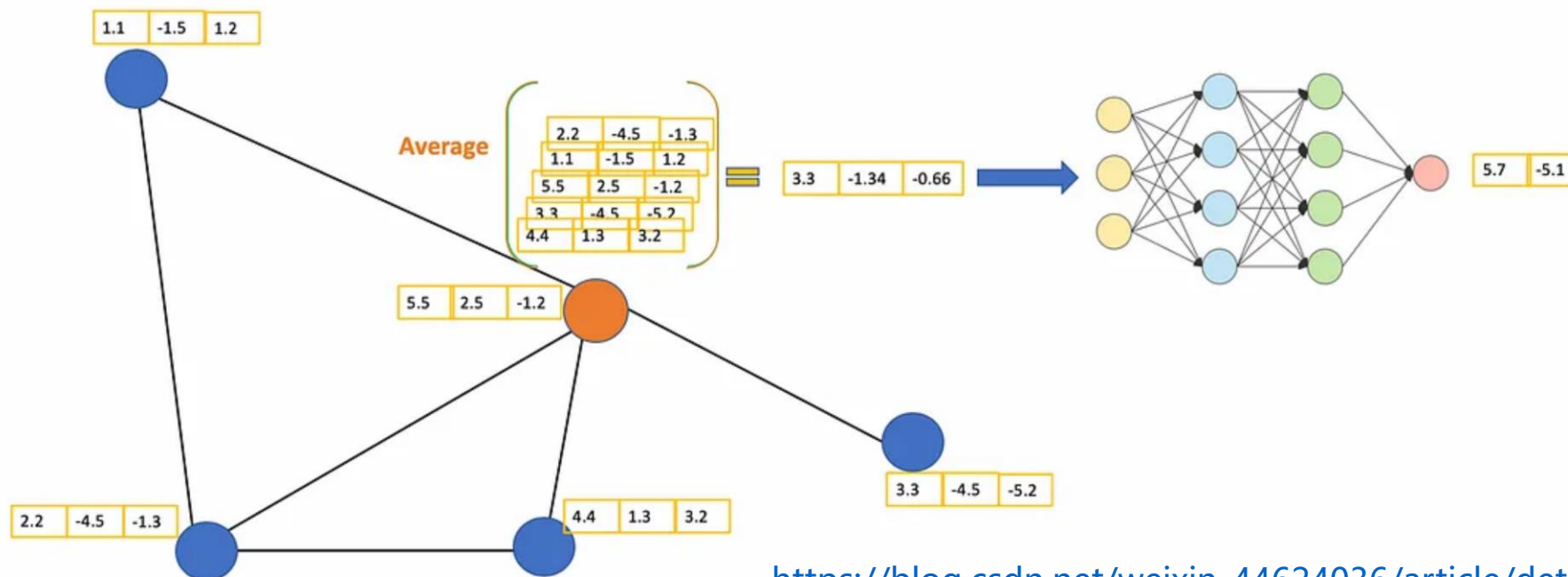
Input: Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times E}$, preprocessed adjacency matrix $\hat{\mathbf{A}}$



[Kipf & Welling, ICLR 2017]



Fundamental idea of GCN



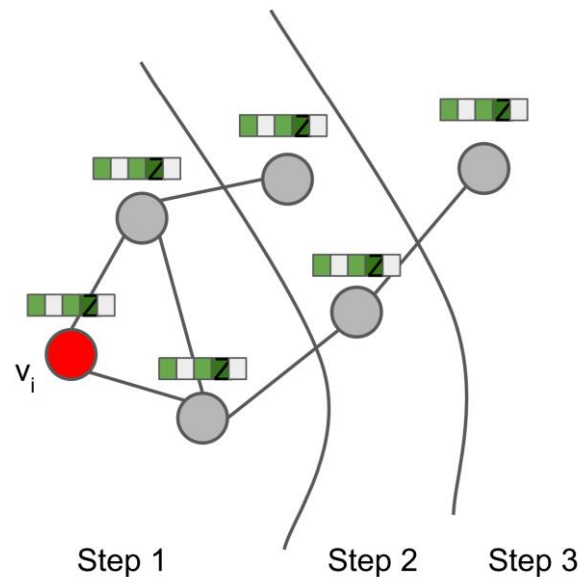
https://blog.csdn.net/weixin_44624036/article/details/131622229

- For the orange node, calculate its features by averaging the features of its neighbors (including itself) and then passing them through a neural network
- Similar to convolutional neural networks (CNNs), GCNs can have multiple layers.
 - In each layer, the input remains the node features
 - These features are then passed to the next layer along with the network structure graph for further computation



Graph view of GCN model

- At every iteration, the model aggregates information from one hop deeper
- The number of layers in GCN
 - In theory, more layers are better
 - But in real-world graphs, you might not need that many layers
 - For example, in social networks you can reach anywhere in the entire world with just 6 hops
 - Therefore, most GCNs do not have many layers
 - Across multiple graph datasets, it has been found that 2 to 3 layers are relatively suitable, and having too many layers may even worsen the performance



Kipf & Welling,
ICLR 2017

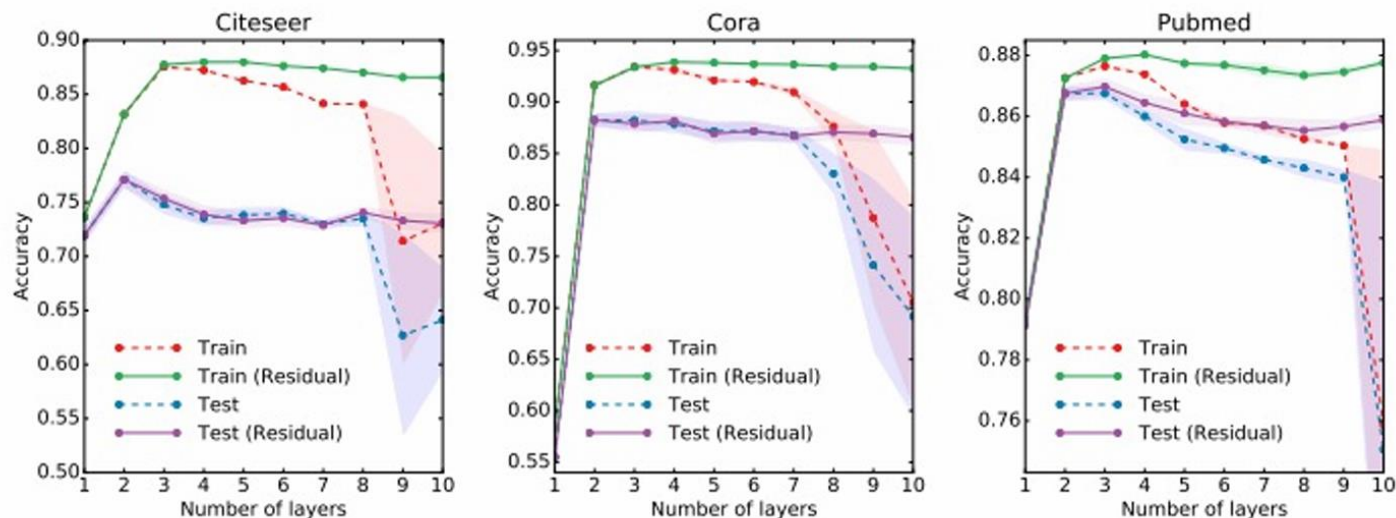


Figure 5: Influence of model depth (number of layers) on classification performance. Markers denote mean classification accuracy (training vs. testing) for 5-fold cross-validation. Shaded areas denote standard error. We show results both for a standard GCN model (dashed lines) and a model with added residual connections (He et al., 2016) between hidden layers (solid lines).



Potential limitations of GCNs



- Fixed aggregation function
- Lack of scalability
- Treats all graph edges equally
- Effective depth is limited





GCN extension: GraphSAGE



SAGE = **S**Ample Aggre**G**at**E**

Algorithm 1: GraphSAGE embedding generation (i.e., forward propagation) algorithm

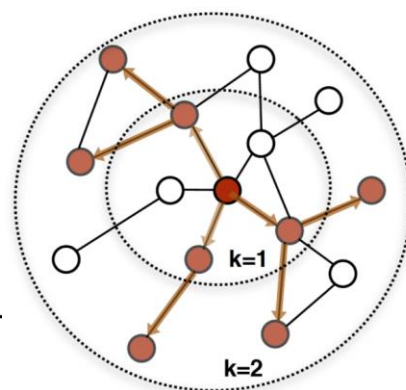
Input : Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; input features $\{\mathbf{x}_v, \forall v \in \mathcal{V}\}$; depth K ; weight matrices $\mathbf{W}^k, \forall k \in \{1, \dots, K\}$; non-linearity σ ; differentiable aggregator functions $\text{AGGREGATE}_k, \forall k \in \{1, \dots, K\}$; neighborhood function $\mathcal{N} : v \rightarrow 2^{\mathcal{V}}$

Output : Vector representations $\mathbf{z}_v$ for all $v \in \mathcal{V}$

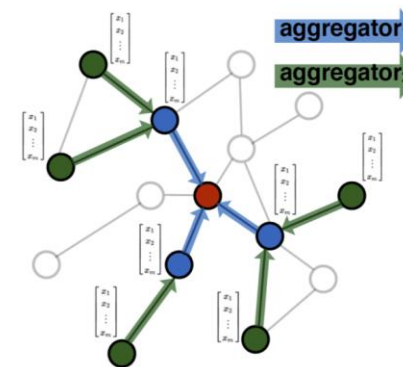
```
1  $\mathbf{h}_v^0 \leftarrow \mathbf{x}_v, \forall v \in \mathcal{V}$ ;  
2 for  $k = 1 \dots K$  do  
3   for  $v \in \mathcal{V}$  do  
4      $\mathbf{h}_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\})$ ;  
5      $\mathbf{h}_v^k \leftarrow \sigma(\mathbf{W}^k \cdot \text{CONCAT}(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^k))$   
6   end  
7    $\mathbf{h}_v^k \leftarrow \mathbf{h}_v^k / \|\mathbf{h}_v^k\|_2, \forall v \in \mathcal{V}$   
8 end  
9  $\mathbf{z}_v \leftarrow \mathbf{h}_v^K, \forall v \in \mathcal{V}$ 
```

Flexible aggregation (e.g. average or LSTM)

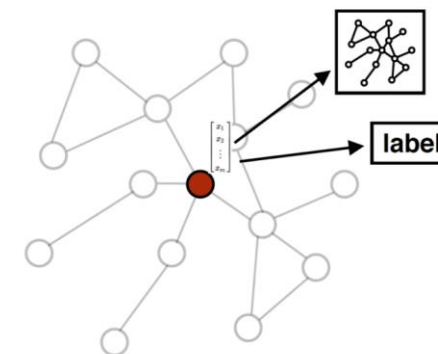
Stochastic sampling



1. Sample neighborhood



2. Aggregate feature information from neighbors

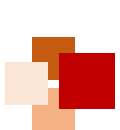


3. Predict graph context and label using aggregated information

Figure 1: Visual illustration of the GraphSAGE sample and aggregate approach.

Hamilton, et al., NIPS 2017

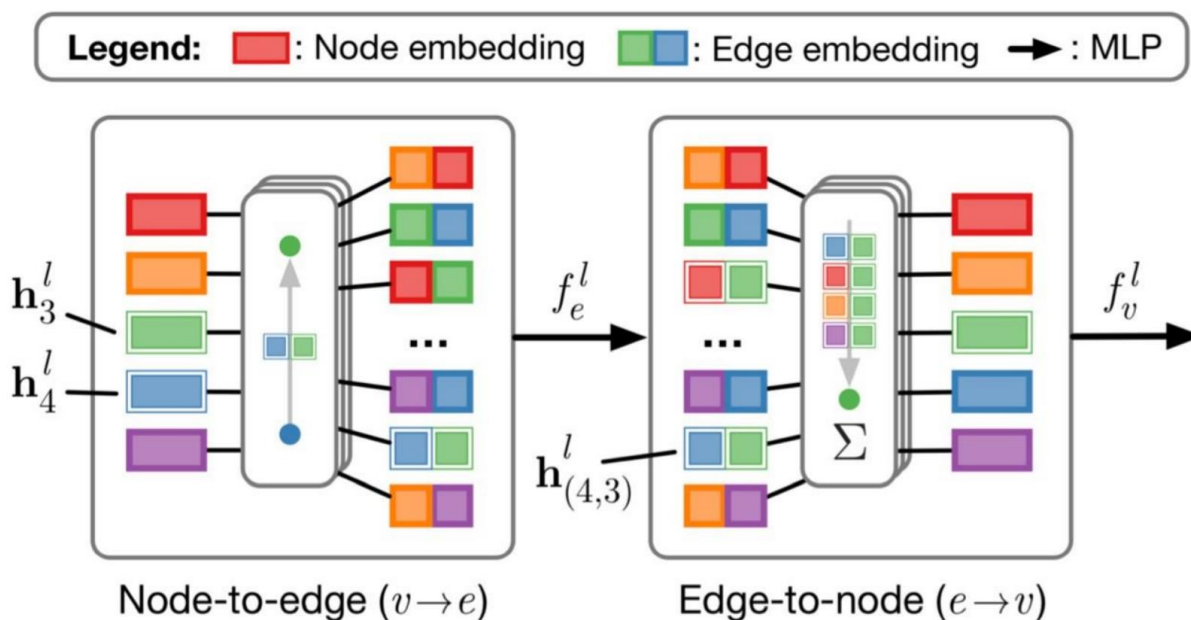




GNNs with edge embeddings



Battaglia et al. (NIPS 2016), Gilmer et al. (ICML 2017), Kipf et al. (ICML 2018)



Pros:

- Supports edge features
- More expressive than GCN
- As general as it gets (?)
- Supports sparse matrix ops

Cons:

- Need to store intermediate edge-based activations
 - Difficult to implement with subsampling
- ➔ In practice limited to small graphs

Formally:

$$v \rightarrow e : \mathbf{h}_{(i,j)}^l = f_e^l([\mathbf{h}_i^l, \mathbf{h}_j^l, \mathbf{x}_{(i,j)}])$$
$$e \rightarrow v : \mathbf{h}_j^{l+1} = f_v^l([\sum_{i \in \mathcal{N}_j} \mathbf{h}_{(i,j)}^l, \mathbf{x}_j])$$

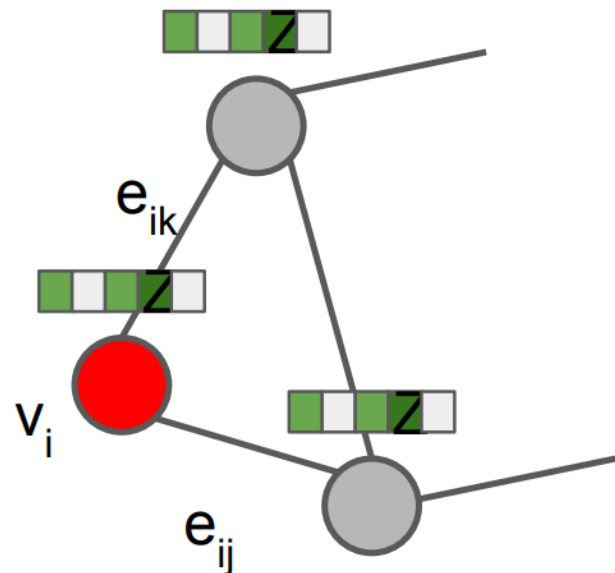


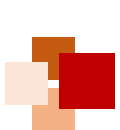
GNNs with Attention (GATs)

- **Problem:** GCNs assume that the importance of the edges in the graph are equal.
- **Idea:** What if we add a **learnable attention weight** α_{ij} for each edge?

$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})}.$$

$$\alpha_{ij} = \frac{\exp \left(\text{LeakyReLU} \left(\vec{\mathbf{a}}^T [\mathbf{W} \vec{h}_i \parallel \mathbf{W} \vec{h}_j] \right) \right)}{\sum_{k \in \mathcal{N}_i} \exp \left(\text{LeakyReLU} \left(\vec{\mathbf{a}}^T [\mathbf{W} \vec{h}_i \parallel \mathbf{W} \vec{h}_k] \right) \right)}$$

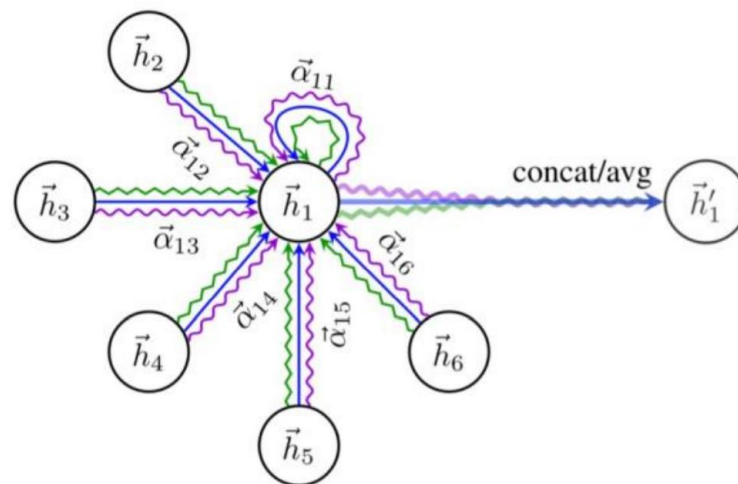
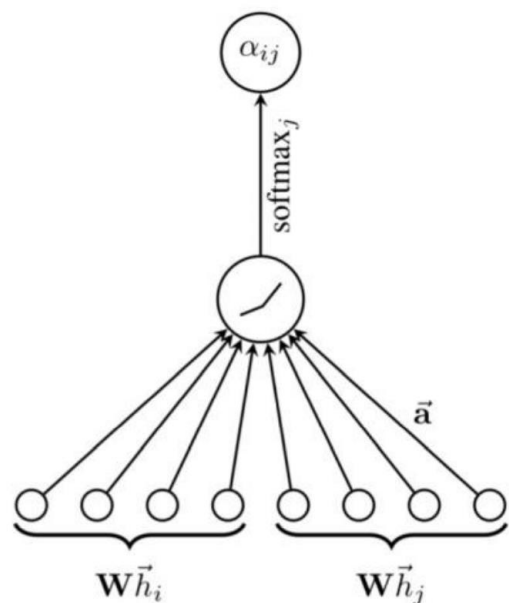




GNNs with Attention (GATs)



Monti et al. (CVPR 2017), Hoshen (NIPS 2017), Veličković et al. (ICLR 2018)



[Figure from Veličković et al. (ICLR 2018)]

Pros:

- No need to store intermediate edge-based activation vectors (when using dot-product attn.)
- Slower than GCNs but faster than GNNs with edge embeddings

Cons:

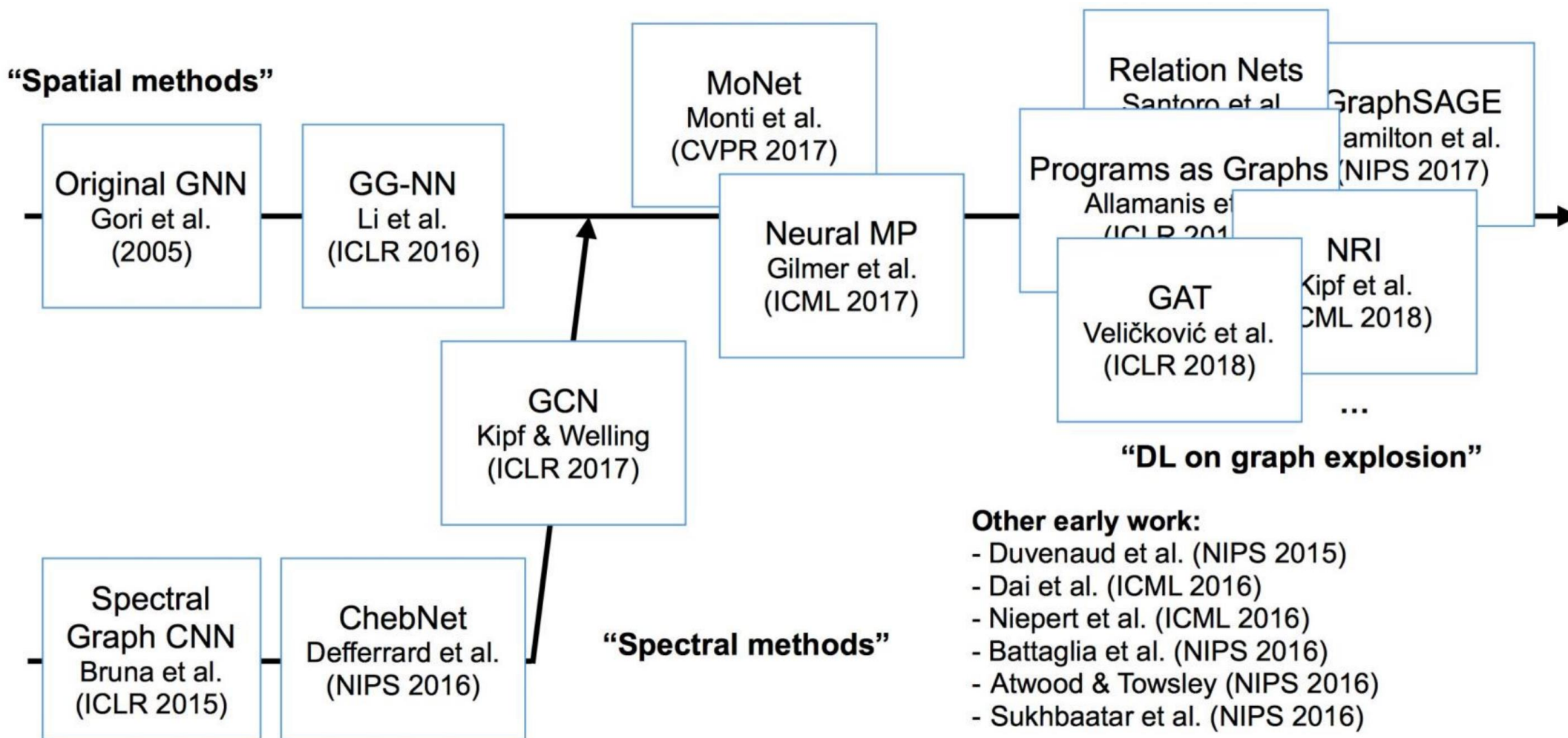
- (Most likely) less expressive than GNNs with edge embeddings
- Can be more difficult to optimize

$$\vec{h}'_i = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k \vec{h}_j \right) \quad \alpha_{ij} = \frac{\exp \left(\text{LeakyReLU} \left(\vec{a}^T [\mathbf{W} \vec{h}_i \| \mathbf{W} \vec{h}_j] \right) \right)}{\sum_{k \in \mathcal{N}_i} \exp \left(\text{LeakyReLU} \left(\vec{a}^T [\mathbf{W} \vec{h}_i \| \mathbf{W} \vec{h}_k] \right) \right)}$$





A brief history of GNNs



(slide inspired by Alexander Gaunt's talk on GNNs)

Slide from Thomas Kipf

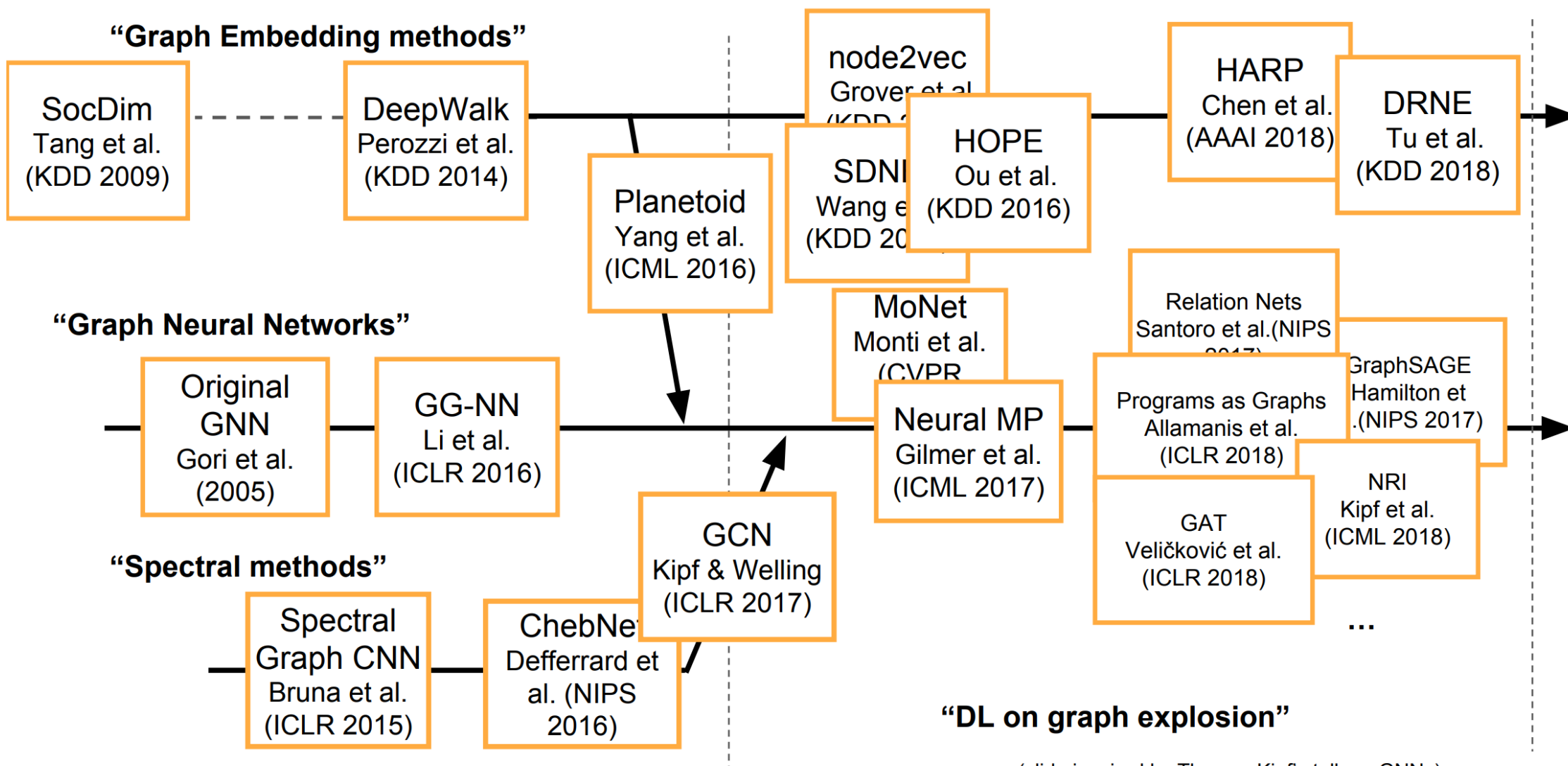




A brief history of GNNs



上海科技大学
ShanghaiTech University



(slide inspired by Thomas Kipf's talk on GNNs)

Slide by Bryan Perozzi (KDD 2018 Tutorial)



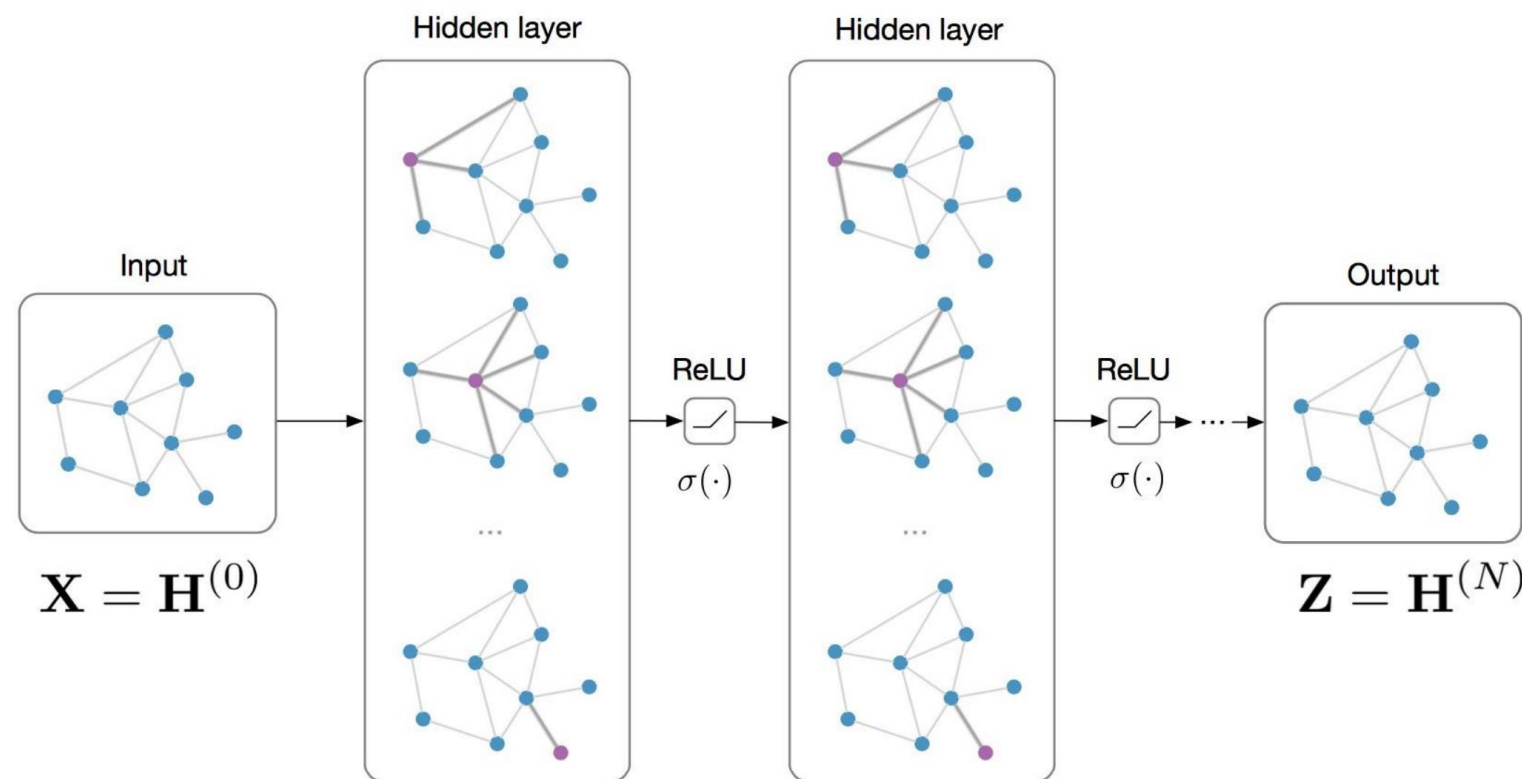
Applications of GNNs



Classification and link prediction with GNNs



Input: Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times E}$, preprocessed adjacency matrix $\hat{\mathbf{A}}$



$$\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{A}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right)$$

Node classification:

$$\text{softmax}(\mathbf{z}_n)$$

e.g. Kipf & Welling (ICLR 2017)

Graph classification:

$$\text{softmax}(\sum_n \mathbf{z}_n)$$

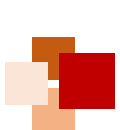
e.g. Duvenaud et al. (NIPS 2015)

Link prediction:

$$p(A_{ij}) = \sigma(\mathbf{z}_i^T \mathbf{z}_j)$$

Kipf & Welling (NIPS BDL 2016)

“Graph Auto-Encoders”



Semi-supervised classification on graphs



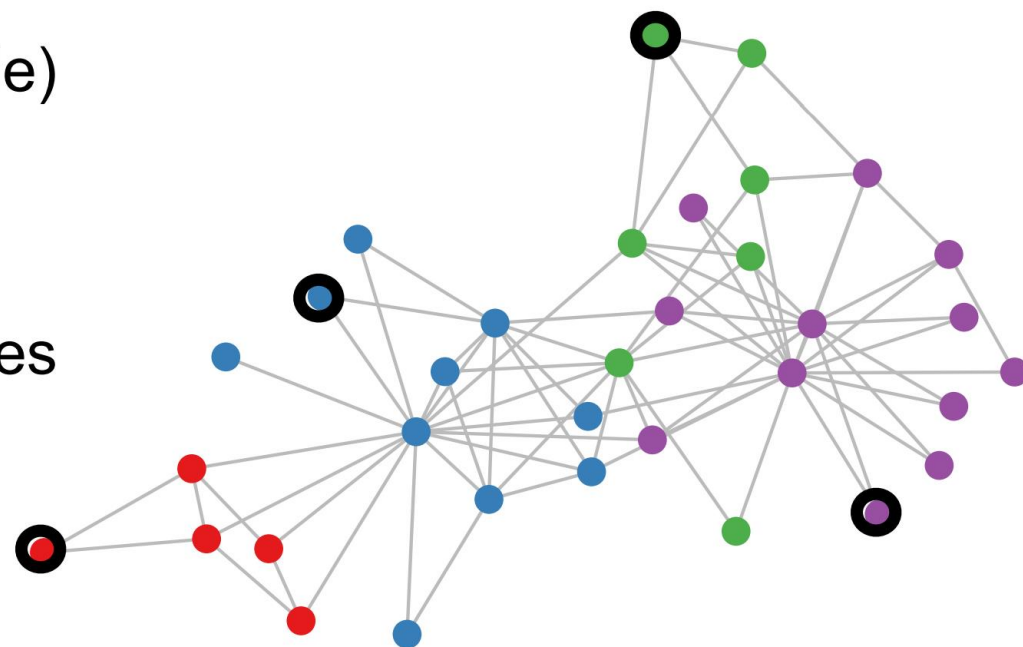
上海科技大学
ShanghaiTech University

Setting:

Some nodes are labeled (black circle)
All other nodes are unlabeled

Task:

Predict node label of unlabeled nodes



Standard approach:

graph-based regularization [Zhu et al., 2003]

$$\mathcal{L} = \mathcal{L}_0 + \lambda \mathcal{L}_{\text{reg}} \quad \text{with} \quad \mathcal{L}_{\text{reg}} = \sum_{i,j} A_{ij} \|f(X_i) - f(X_j)\|^2$$

assumes: connected nodes likely to share same label





Semi-supervised classification on graphs



- **Embedding-based approaches**
- Two-step pipeline:
 - 1) Get an embedding for every node
 - 2) Train a classifier on the node embedding
- **Examples:** DeepWalk [Perozzi et al., 2014], node2vec [Grover & Leskovec, 2016]
- **Issue:** Embeddings are not optimized for classification
- **Idea:** Train a graph-based classifier end-to-end using GCN

Evaluate loss on labeled nodes only:

$$\mathcal{L} = - \sum_{l \in \mathcal{Y}_L} \sum_{f=1}^F Y_{lf} \ln Z_{lf}$$

$\mathcal{Y}_L$ set of labeled node indices

$\mathbf{Y}$ label matrix

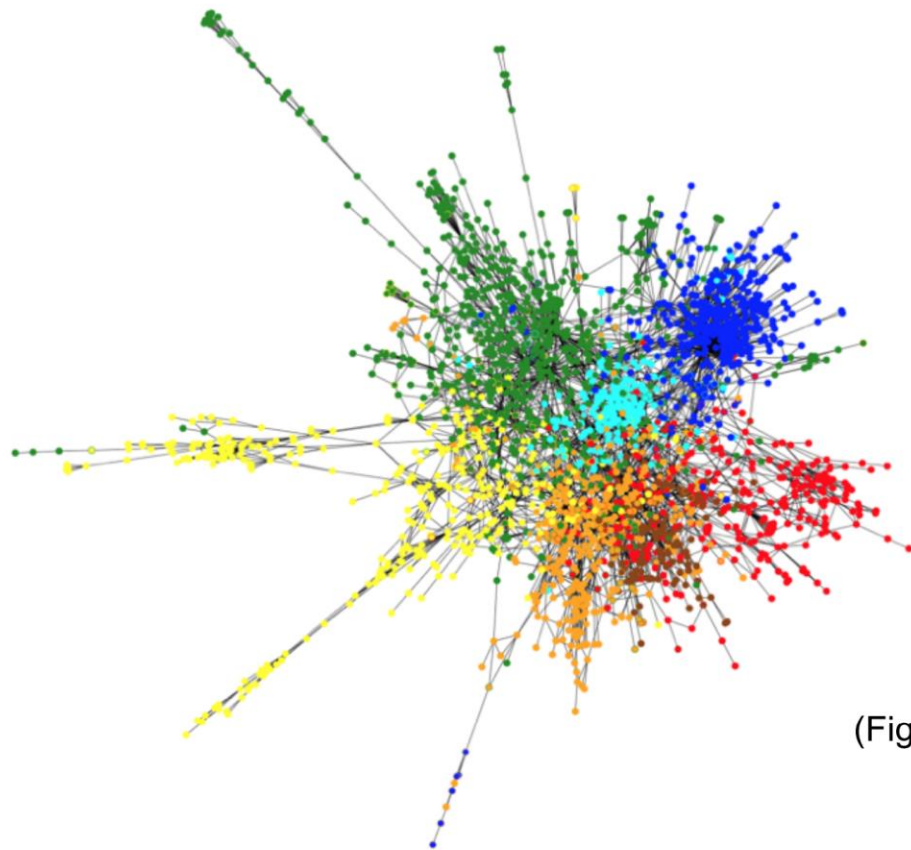
$\mathbf{Z}$ GCN output (after softmax)



Application: Classification on citation networks



- **Input:** Citation networks (nodes are papers, and edges are citation links, optionally with bag-of-words features on the nodes)
- **Target:** Paper category (e.g. stat.ML, cs.LG, ...)



(Figure from: Bronstein, Bruna, LeCun, Szlam, Vandergheynst, 2016)



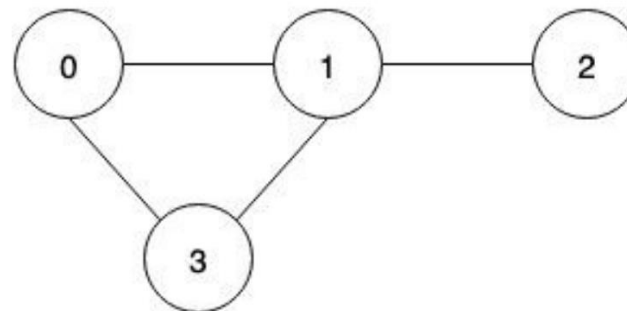


Variational Graph Auto-Encoders (VGAEs)

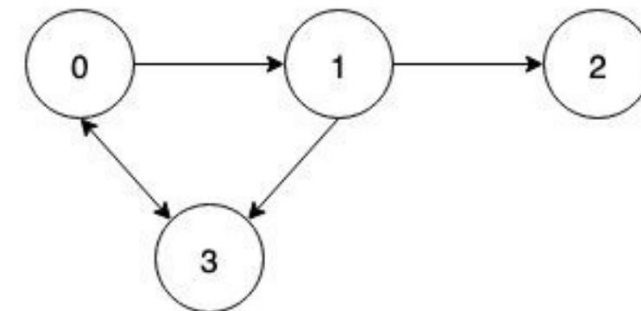


上海科技大学
ShanghaiTech University

- **Main idea:** To apply the idea of Variational Autoencoder (VAE) to graph-structured data, to generate new graphs or reason about graphs (Kipf & Welling, NIPS Bayesian Deep Learning Workshop, 2016)

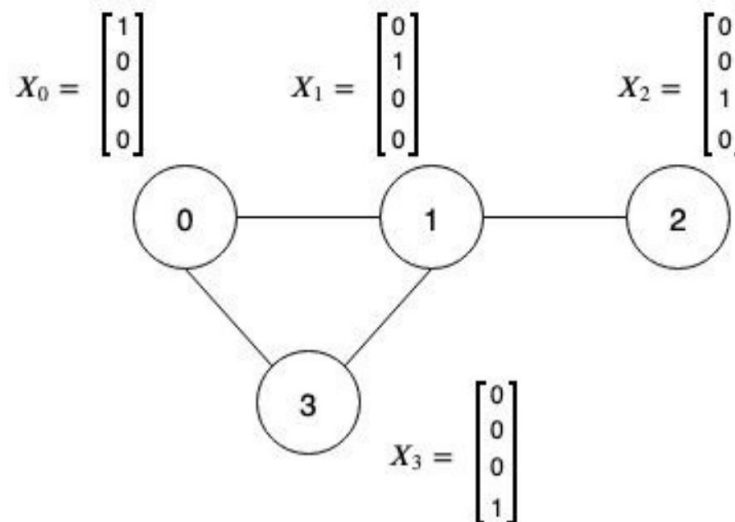


$$A_1 = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$



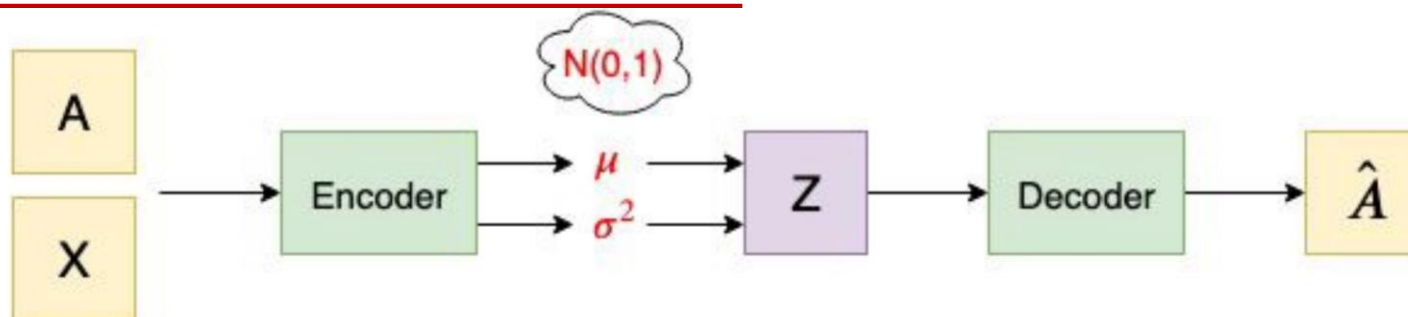
$$A_2 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

- Input data:
 - **Adjacency matrix A :** topology of the graph
 - **Feature matrix X :** features of each node from the input graph



$$X = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Architecture of VGAEs



- **Encoder (inference model):** A 2-layer graph convolutional network (GCN)
- GCN layer 1 outputs a lower-dimensional feature matrix: $\tilde{X} = GCN(X, A) = ReLU(\tilde{A}XW_0)$

$$\tilde{A} = D^{-\frac{1}{2}}AD^{-\frac{1}{2}}$$

- GCN layer 2 outputs the two parameters of a Normal distribution:

$$GCN(X, A) = \tilde{A}ReLU(\tilde{A}XW_0)W_1$$

$$\mu = GCN_{\mu}(X, A)$$

$$\log\sigma^2 = GCN_{\sigma}(X, A)$$

- **Decoder (generative model):** An inner product between latent variable Z and its transpose, which is a reconstructed adjacency matrix $\hat{A}$

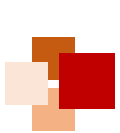
$$\hat{A} = \sigma(zz^T)$$

Calculate Z using parameterization trick

$$Z = \mu + \sigma * \epsilon$$

where $\epsilon \sim N(0, 1)$

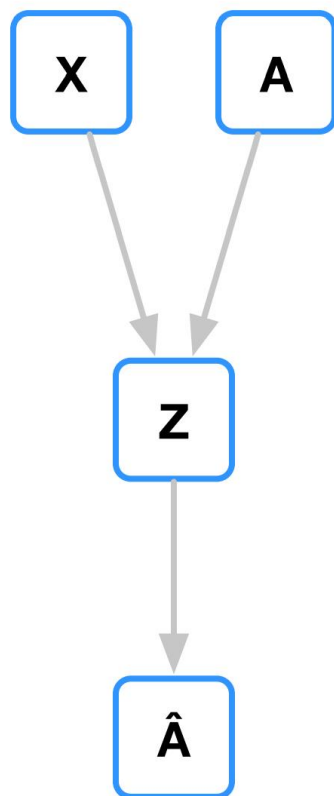
<https://towardsdatascience.com/tutorial-on-variational-graph-auto-encoders-da9333281129>



Link prediction with GAEs

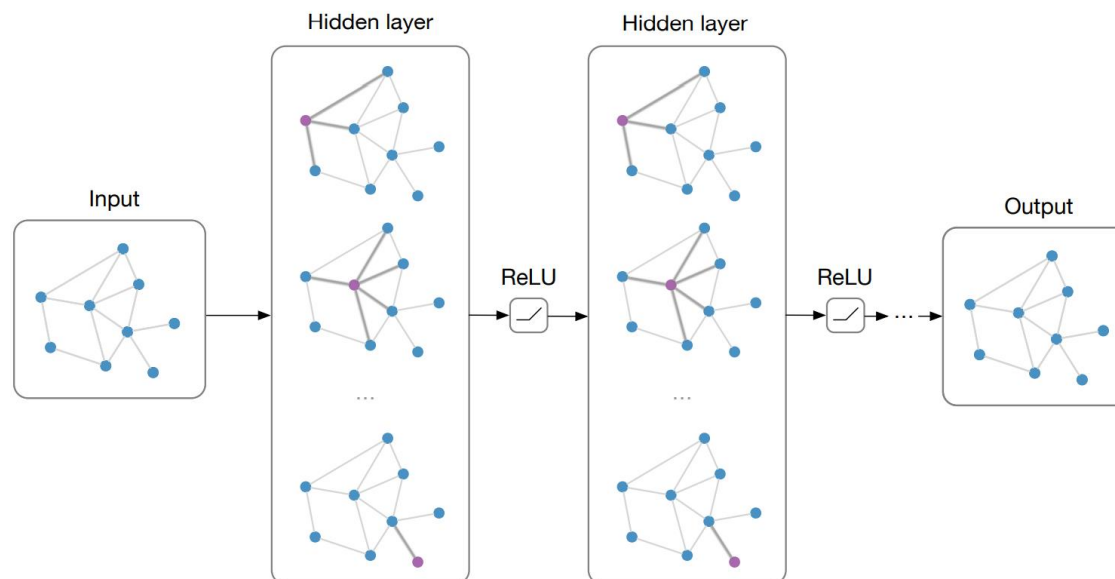


Kipf & Welling, NIPS Bayesian Deep Learning Workshop, 2016



GAE

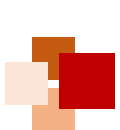
Encoder $\mathbf{Z} = \text{GCN}(\mathbf{X}, \mathbf{A})$



Decoder $\hat{\mathbf{A}} = \sigma(\mathbf{Z}\mathbf{Z}^\top)$

This is a **non-probabilistic graph auto-encoder (GAE)** model, a variant of VGAE



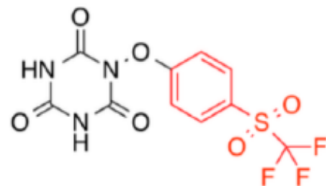
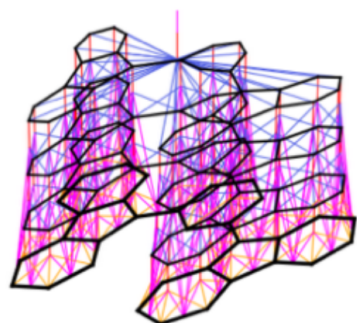


Other recent applications



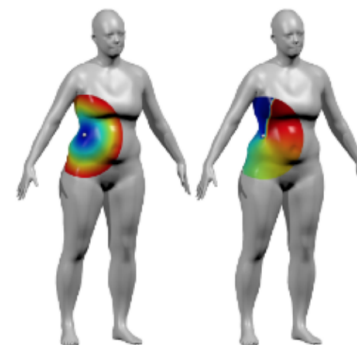
上海科技大学
ShanghaiTech University

Molecules

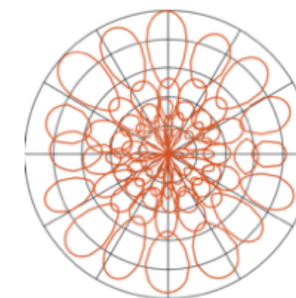


[Duvenaud et al., NIPS 2015]

Shapes



Polar coordinates ρ, θ



MoNet

[Monti et al., 2016]

Knowledge Graphs



[Schlichtkrull et al., 2017]





Summary



- In this lecture we have learned
 - The motivation for GNNs is the existence of lots of graph-structured data, on which standard CNNs or RNNs can't work
 - Graph convolutional networks (GCNs) were basically about applying ideas of CNNs to graphs, but have their own explanation (e.g. message passing and aggregation)
 - There are some powerful extensions of GCNs, e.g. GraphSAGE, edge embeddings, GNNs with attention
 - GNNs have mainly 3 categories of application tasks: node classification, graph classification, and link prediction
 - Graph Auto-encoders (GAEs) can do link prediction
 - There are many applications of GNNs, e.g. molecules, drug discovery, knowledge graph, etc.





References



- Scarselli et al., "The Graph Neural Network Model" , IEEE Transactions on Neural Networks, 2009.
- Bruna et al., "Spectral Networks and Deep Locally Connected Networks on Graphs" , ICLR 2014.
- Kipf and Welling, "Variational Graph Auto-Encoders" , NIPS Bayesian Deep Learning Workshop, 2016.
- Kipf and Welling, "Semi-Supervised Classification with Graph Convolutional Networks" , ICLR 2017.
- Hamilton et al., "Inductive Representation Learning on Large Graphs" , NIPS 2017.
- Veličković et al., "Graph Attention Networks" , ICLR 2018.
- Kipf et al., "Neural Relational Inference for Interacting Systems" , ICML 2018.
- Wu et al., "A Comprehensive Survey on Graph Neural Networks" , IEEE Transactions on Neural Networks and Learning Systems, 2020.
- Sun et al., "Graph Convolutional Network for Computational Drug Development and Discovery" , Briefings in Bioinformatics, 2020.

